The covering number $C(13, 7, 4)$ is 30

A classification-assisted exact proof

John Tripodo
ORCID: 0009-0006-9180-4953

29 October 2026
Public proof-source edition; preprint for review

Abstract

We prove that $C(13, 7, 4) = 30$, using the published classification of minimum $(12, 6, 3)$ coverings. Incidence counting forces a hypothetical 29-block cover to have at least five points of degree 15. The link at one such point is classified, while the other four force complete families of outside blocks with mutually consistent overlaps. There are only 14 outside blocks available. An exact enumeration over all $68\binom{12}{4} = 33{,}660$ labelled root configurations finds that every compatible four-pencil tuple requires 15 outside blocks, a contradiction. An explicit known 30-block cover supplies the upper bound. We provide two separately structured Python verifiers and a public Lean formalization of a second, actual-cover certificate argument. The Lean endpoint has the published minimum bound and classification as explicit hypotheses; its internal finite census is fully discharged. The released source closure, dependency pins, exact theorem correspondence and historical axiom audit make that boundary inspectable. The 1995 classification is not re-proved in this work.

Keywords: covering designs; covering numbers; finite classification; exact enumeration; computational proof; Lean formalization.

1 The result and its context

For positive integers $v \geq k \geq t$, a (v, k, t) *covering* is a family $\mathcal{F} \subseteq \binom{X}{k}$, where $|X| = v$, such that every $T \in \binom{X}{t}$ is contained in some $B \in \mathcal{F}$. Its minimum cardinality is the covering number $C(v, k, t)$. The problem considered here asks for the minimum number of seven-element blocks on thirteen points needed to cover every four-element subset.

Theorem 1.1 (Main result)**.**

$$\boxed{C(13, 7, 4) = 30.}$$

An explicit construction supplies the upper bound 30; the contribution is the global exclusion of 29 blocks. Prior tabulations gave the interval $28 \leq C(13, 7, 4) \leq 30$ [2], and a separately published, formally checked elimination of one minimum-link type does not resolve all 68 types [3]. The present argument eliminates all types, not merely one restricted instance. Related recent work on a different covering number illustrates the increasing role of independently checked finite certificates in design theory [4].

The one external mathematical ingredient is the work of Gordon, Patashnik, Petro and Taylor [1]: their Theorem 4.4 proves $C(12, 6, 3) = 15$, and their Theorem 5.9 classifies the minimum

$(12, 6, 3)$ covers by a Figure 1 completion family and a single exceptional Figure 6 cover. Their 68 nonisomorphic minimum covers are *input*, not an original classification claimed here. The proof below explains why that classification is enough and isolates the precise role of the computation.

Proof in one paragraph. Every point of a hypothetical 29-block covering occurs in at least 15 blocks, and there are only eight degree units above that baseline; hence at least five points have degree exactly 15. Root at one such point. Its 15 incident blocks determine a classified link, leaving just 14 blocks not containing the root. Each of four further minimum points forces a full set of outside blocks. The classification supplies exhaustive finite domains for those sets. The computation proves that four mutually compatible choices always need at least 15 outside blocks. This is the contradiction.

2 Why five minimum points are unavoidable

Write $X = Y \cup \{\infty\}$ with $|Y| = 12$ once a root is chosen. For any point $x \in X$, define its degree and link by

$$d_{\mathcal{F}}(x) := |\{B \in \mathcal{F} : x \in B\}|, \qquad L_x(\mathcal{F}) := \{B \setminus \{x\} : B \in \mathcal{F},\ x \in B\}.$$

If $\mathcal{F}$ covers every four-set of X, then $L_x(\mathcal{F})$ covers every three-set of $X \setminus \{x\}$. The cited minimum-cover theorem therefore implies $d_{\mathcal{F}}(x) \geq 15$ for every x.

Lemma 2.1 (Five minimum points)**.** *If a* $(13, 7, 4)$ *covering has exactly* 29 *distinct blocks, at least five points have degree* 15*.*

Proof. Double-count incidences between points and blocks:

$$\sum_{x \in X} d_{\mathcal{F}}(x) = 7|\mathcal{F}| = 7 \cdot 29 = 203 = 13 \cdot 15 + 8.$$

Each degree exceeds 15 by a nonnegative integer. At most eight points can have positive excess, leaving at least five of degree 15. □

Choose such a point ∞. The root link $L := L_\infty(\mathcal{F})$ has exactly 15 blocks and is one of the published minimum $(12, 6, 3)$ covers up to relabelling. The *outside family*

$$\mathcal{F}^\circ := \{B \in \mathcal{F} : \infty \notin B\}$$

contains exactly $29 - 15 = 14$ blocks, all seven-subsets of Y. Choose four other degree-15 points $Q \subseteq Y$, $|Q| = 4$. We now show how they constrain $\mathcal{F}^\circ$.

Why use several minimum points? A single minimum point gives a 15-block link but leaves a difficult extension problem on the other twelve points. Every additional minimum point specifies *all* outside blocks through that point. Consequently these local specifications must agree on every shared block. Their union gives a lower bound on the number of outside blocks. The useful obstruction here is the gap $15 > 14$ obtained from four such specifications. It is an obstruction to compatible local links, not a new construction of the upper-bound cover.

3 From classified links to complete outside pencils

Let $\mathcal{C}$ be a set of representatives of the 68 published isomorphism classes of minimum $(12, 6, 3)$ coverings, each with 15 distinct six-subsets of a labelled twelve-element set. Fix a labelled root link $L \in \mathcal{C}$. For $q \in Y$, the root-containing part of the q-link is already fixed:

$$S_L(q) := \{\{\infty\} \cup (A \setminus \{q\}) : A \in L,\ q \in A\}. \tag{1}$$

This is a family of six-subsets of $X \setminus \{q\}$.

Take a classified source link $K \in \mathcal{C}$ on a twelve-point set W, a source point $c \in W$, and a bijection $\phi : W \to X \setminus \{q\}$ such that $\phi(c) = \infty$ and

$$\{\phi(A) : A \in K,\ c \in A\} = S_L(q). \tag{2}$$

Define the associated *complete outside pencil*

$$P(L, q; K, c, \phi) := \{\{q\} \cup \phi(A) : A \in K,\ c \notin A\} \subseteq \binom{Y}{7}. \tag{3}$$

The domain $D_L(q)$ consists of all pencils in (3) over all K, c, ϕ satisfying (2). In computation we may first prune c by the necessary equality $d_K(c) = d_L(q)$, but the definition of the domain itself is unrestricted.

Lemma 3.1 (Domain completeness)**.** *Suppose $\mathcal{F}$ is a 29-block $(13, 7, 4)$ covering with root link L and $d_{\mathcal{F}}(q) = 15$. Then*

$$P_q := \{B \in \mathcal{F}^\circ : q \in B\} \in D_L(q).$$

In particular, P_q records every *outside block through q, not an arbitrarily chosen subfamily.*

Proof. The full link $L_q(\mathcal{F})$ is a minimum $(12, 6, 3)$ cover. The cited classification gives a source $K \in \mathcal{C}$ and a bijection from its twelve points onto $X \setminus \{q\}$ carrying K to $L_q(\mathcal{F})$. If c maps to ∞, the blocks of K containing c map precisely to the known star $S_L(q)$. The remaining blocks of K are exactly the blocks of the q-link that avoid ∞. Reattaching q to their images gives (3), which is exactly P_q. □

The word *unrestricted* in this construction is essential. The full map on the other eleven source points is searched, subject to exact star equality. Restricting ϕ to a convenient automorphism subgroup would not by itself justify exhaustiveness.

Equivariance of domain reuse. The implementations may reuse a domain when two known stars are isomorphic. This does not require choosing every isomorphism between the target stars. If g is one such isomorphism, extend it by mapping the distinguished target point to the new distinguished target point. Composition with g bijects every classified source embedding for the first star with an embedding for the second, with inverse composition by g^{-1}. Applying this bijection to all complete pencils proves that the transported domain is the entire second domain. Equality of coarse invariants alone is never used to authorize transport.

4 Agreement conditions and the finite obstruction

For distinct $q, r \in Q$ and pencils $P \in D_L(q)$, $R \in D_L(r)$, call (P, R) *compatible* when

$$\{B \in P : r \in B\} = \{B \in R : q \in B\}. \tag{4}$$

The two sides are the outside seven-blocks through both points. Thus the actual pencils P_q, P_r supplied by Lemma 3.1 are pairwise compatible. Also, because each $P_q \subseteq \mathcal{F}^\circ$,

$$\left| \bigcup_{q \in Q} P_q \right| \leq |\mathcal{F}^\circ| = 14. \tag{5}$$

The central finite statement is the following.

Proposition 4.1 (Exhaustive four-pencil obstruction)**.** *For every $L \in \mathcal{C}$ and every $Q \in \binom{Y}{4}$, there are no choices $P_q \in D_L(q)$, $q \in Q$, which satisfy every pair compatibility equation* (4) *and the union budget* (5)*.*

Exact computer-assisted verification. The exhaustive input space contains $68\binom{12}{4} = 68 \cdot 495 = 33{,}660$ *labelled* pairs (L, Q). No orbit reduction of the Q-sets is needed for this claim.

The computation first rejects an input if some pair of selected points has no compatible pair of possible pencils. It then enforces arc consistency: any pencil with no compatible partner in another selected domain is removed, and this rule repeats until stable. These are sound necessary filters. To see the latter formally, consider a putative compatible four-pencil choice. Initially each actual choice lies in its full domain. If all four remain after a round, each is a compatible partner for each of the other three, so none can be removed in the next round. By induction a genuine compatible choice is never eliminated.

The recorded exact census is:

Exhaustive stage	Labelled root/fourset inputs
A pair has no compatible pencil choices	31,263
Additional inputs closed by sound arc consistency	2,355
Inputs retained for the final four-way join	42
Total: $68\binom{12}{4}$	33,660

Table 1: Disjoint partition of the complete Python/C++ finite search. The last line is an exhaustive total, not an extrapolation from representative foursets.

For each of the 42 residual inputs, a separately written routine constructs all six pair-compatibility relations and enumerates every four-pencil tuple satisfying them. Importantly, this final routine *does not prune by union size*; it computes the literal union after joining. Its result is:

The reconstruction of domains uses the 68-link catalog, with an independent route rebuilding all $28^3 = 21{,}952$ literal Figure-1 completions from the published core and short blocks, recovering 67 completion classes under the construction action, and separately including the exceptional Figure-6 link. Both implementations reconstruct $68 \cdot 12 = 816$ root/point domains, agree on the canonical digest of every domain, and obtain Tables 1 and 2. Python ordinary and optimized executions agree; a distinct C++ implementation reproduced the decisive totals. The source, manifests, independent joins and build receipts are itemized in Appendix B. No floating-point optimization status, solver timeout or unverified infeasibility declaration enters the conclusion. Every residual compatible union has $15 > 14$ blocks, so (5) is impossible. □

Root type	Residual inputs	Compatible tuples per input	Total tuples
Figure 1 completion types	37	216	7,992
Exceptional Figure 6 type	5	54,432	272,160
Total	42		280,152

Table 2: Independent unpruned join. Every one of the 280,152 compatible tuples has a union of exactly 15 distinct outside blocks.

5 Proof of optimality

Proof of Theorem 1.1. Suppose a covering has at most 29 blocks. Removing duplicate blocks, if any, preserves coverage; we can pad with previously unused seven-subsets to obtain a covering with exactly 29 distinct blocks, since $\binom{13}{7} = 1716$.

By Lemma 2.1, choose five degree-15 points. Relabel one as ∞ and normalize its minimum link to a representative $L \in \mathcal{C}$; this induces a relabelling of the twelve ordinary points. Choose the other four minimum points as Q. Lemma 3.1 supplies one complete outside pencil $P_q \in D_L(q)$ for each $q \in Q$. All six compatibility equations hold, and the pencils jointly occupy at most the 14 outside blocks. This contradicts Proposition 4.1. Therefore $C(13, 7, 4) \geq 30$. The explicit 30-block covering in Appendix A, checked against every one of the $\binom{13}{4} = 715$ four-subsets, supplies $C(13, 7, 4) \leq 30$. □

6 A public Lean proof relative to the published classification

The repository accompanying this paper contains the complete local source closure for a second proof of the lower bound. This proof works directly with actual covering families. It does not use the Python verifier as an oracle and does not merely import a Boolean claim that the Python search succeeded.

6.1 The mathematical interface

The ground type `C13.Point` is `Fin 13`. A family is a finite set of finite subsets. The predicate `IsCoverOn U F k t` requires every block of F to be a k-subset of U and every t-subset of U to lie in a block. Thus the formal conclusion concerns the same simple covering number as Theorem 1.1; duplicate blocks are not counted.

The principal endpoint is

```
C13.Global.exact30_after_published_results.
```

It proves the following implication, expressed here without Lean syntax:

$$\begin{aligned} &\mathsf{PublishedMinimumBound} \ \wedge \ \mathsf{PublishedClassification1995} \\ \Longrightarrow \quad &(\exists F \text{ a } (13,7,4) \text{ cover}, \ |F| = 30) \ \wedge \ (\forall F \text{ a } (13,7,4) \text{ cover}, \ |F| \geq 30)\,. \end{aligned}$$

The first hypothesis is the lower-bound part of the published Theorem 4.4. The second is the relabelling coverage supplied by Theorem 5.9 together with the minimum value from Theorem 4.4. They are ordinary theorem hypotheses, not additional Lean axioms. They are nevertheless substantive mathematical assumptions. An axiom audit cannot discharge an explicit hypothesis.

The classification predicate is stated using the literal Figure 1 completion family and Figure 6. The source-to-catalog theorem `catalog_covers_all_raw_sources` then covers every one of the $28^3 = 21{,}952$ completion triples and the exceptional source. The needed direction is coverage by the catalog. Pairwise nonisomorphism of catalog entries under every permutation is not needed for the lower bound.

6.2 The actual-cover certificate argument

For a catalog link K and a four-set Q, the predicate `ActualObstruction K Q` excludes an actual global cover whose root link is K, whose points in Q have degree 15, and whose outside family has at most 14 blocks. The definition retains the classification hypothesis. It is not the stronger statement that every abstract assignment of local pencils is impossible without the global covering constraints.

The formal census uses the same 33,660 labelled positions, with the following distinct partition:

Formal exclusion mechanism	Positions
Empty-support certificates	31,762
Exact integer-dual certificates and checked transports	1,893
Exceptional Figure-6 predecessor certificates	5
Total	33,660

This is not the Python partition of Table 1. The integer-dual branch contains 964 representative positions and 929 positions reached by checked root automorphisms. The empty-support branch uses 3,636 certificates, some of which exclude more than one four-set.

The formal argument includes the full $792 = \binom{12}{7}$ outside block universe, complete-coverer masks, and all $816 = 68 \cdot 12$ root/point alphabet bindings. A block containing none of the selected points is handled separately; it cannot be silently discarded from the outside-block budget. Row-validity theorems derive the covering and incidence constraints from genuine covers before a dual inequality is used. The checked census and `all_catalog_foursets` feed into `no_cover29`; the final theorem combines this with padding and the unconditional 30-block witness.

6.3 Paper-to-Lean correspondence

The following modules are under `lean/` in the public repository. The correspondence is semantic; it does not assert that the Lean certificate search follows the Python search instruction for instruction.

Mathematical step	Formal source
Links, degrees, incidence sum, five minimum points, padding	`C13Covering/Reduction.lean`
Literal historical classification predicate	`C13Covering/SourceFigures.lean`
Coverage of all raw classified sources by the catalog	`C13Covering/OrbitAllSources.lean`
Complete semantic alphabets and unrestricted star transport	`C13RawStarRecovery/StarTranslation.lean`
Actual-cover obstruction, covering of all four-sets, transport	`C13Global/CensusCore.lean`
Validity of exact covering/degree rows and dual implications	`C13Global/RowValidity.lean`, `C13Global/DualBridge.lean`
All 68 roots; no 29-cover; exact-covering endpoint	`C13Global/Final.lean`
Unconditional 30-block witness and model transport	`C13/Known30.lean`, `C13Covering/PositiveBridge.lean`

An additional endpoint, `exact30_after_published_classification`, derives the minimum-bound parameter from the stated all-size-15 classification predicate and checked private-triple irredundancy of every catalog block. This is a valid implication within that formal interface. It is not a claim that the historical proof of Theorem 5.9 is independent of Theorem 4.4, nor a new proof of the 1995 classification.

7 Reproducibility, assurance and limitations

7.1 Public source and exact identities

The paper, formalization, verification programs, and versioned manifests are available at

`https://github.com/jjtripodo-lgtm/c13-covering-number`.

The immutable proof-source snapshot is commit `254ee82fdbc2`. The Lean export contains 1,751 local source modules. Each file is bound to its original Git blob and SHA-256 digest in `verification/lean_source_manifest.json`. The export changes package layout and build configuration, not theorem source bytes. All local imports resolve inside the public package. External dependencies are revision-pinned; no private repository is needed. The release identity and the tests actually performed are recorded separately from the inherited build evidence.

7.2 Verification layers

The Python proof reconstructs the domains in two different ways, compares all 816 normalized domains, and runs the unpruned compatible-tuple join. Normal and optimized Python runs are compared with frozen receipts. The source packet also preserves a distinct native C++ replay. The known upper-bound cover is checked on all 715 quadruples; corrupt-input and brute-force differential controls exercise both domain generation and the join.

The historical Lean audit checked 76,982 imported project declarations in 1,750 modules. Its allowed axiom set is `propext`, `Classical.choice`, and `Quot.sound`; unsafe declarations and other axioms are rejected. The exact compressed audit log is public. A later recorded replay rechecked the endpoint and negative controls with source/object-bound cached dependencies; it did not rebuild

all imported proofs. Neither record is relabelled as a fresh build of this newly packaged public repository. The release verifier authenticates the historical log and checks the exported source identities; it does not itself execute Lean. The pinned build instructions allow that check to be repeated in a Lean-equipped environment.

7.3 Source and review boundaries

The published $C(12, 6, 3) = 15$ theorem and full link classification remain external mathematical inputs. The author-posted preprint was checked for the relevant theorem wording in preparing this edition; its Figure 1/Figure 6 literal transcriptions remain explicitly available for independent comparison. A source hash authenticates a transcription, not its fidelity to a historical figure. The publication audit distinguishes historical source checking from fresh computational replay.

The manuscript, code and formalization were developed with substantial OpenAI ChatGPT and Codex assistance. The author directs the project; no part is represented as having been checked and understood by a human. Independently structured programs and automated audits are not independent human refereeing. No external specialist review, exhaustive worldwide priority clearance, or journal acceptance is claimed. This result determines one covering number; it does not classify all optimal 30-block covers.

A An explicit optimal 30-block covering

The blocks below are the published Nurmela–Östergård upper-bound construction tabulated by the La Jolla Covering Repository [2], not a new construction claimed here. They are seven-subsets of $\{1, 2, \ldots, 13\}$. Each four-subset is contained in at least one of the 30 blocks. The accompanying independent standard-library checker enumerates all 715 possible four-subsets, verifies distinctness and confirms $1{,}050 = 30\binom{7}{4}$ total block/fourset incidences.

Block	Block
$\{1, 2, 5, 6, 7, 8, 9\}$	$\{1, 3, 4, 5, 6, 8, 10\}$
$\{2, 3, 6, 7, 8, 9, 10\}$	$\{2, 3, 4, 5, 8, 9, 11\}$
$\{1, 3, 5, 7, 8, 10, 11\}$	$\{1, 2, 4, 8, 9, 10, 11\}$
$\{1, 2, 3, 5, 7, 10, 12\}$	$\{1, 3, 4, 5, 9, 10, 12\}$
$\{1, 2, 4, 5, 6, 11, 12\}$	$\{1, 3, 6, 7, 9, 11, 12\}$
$\{2, 6, 7, 8, 9, 11, 12\}$	$\{4, 6, 7, 8, 9, 11, 12\}$
$\{2, 3, 4, 6, 10, 11, 12\}$	$\{5, 6, 7, 9, 10, 11, 12\}$
$\{3, 4, 5, 6, 7, 9, 13\}$	$\{1, 2, 3, 5, 6, 10, 13\}$
$\{1, 4, 6, 7, 9, 10, 13\}$	$\{1, 2, 3, 4, 7, 11, 13\}$
$\{2, 4, 7, 8, 9, 11, 13\}$	$\{2, 4, 5, 7, 10, 11, 13\}$
$\{1, 3, 5, 9, 10, 11, 13\}$	$\{1, 4, 5, 7, 8, 12, 13\}$
$\{2, 4, 6, 7, 8, 12, 13\}$	$\{1, 2, 3, 8, 9, 12, 13\}$
$\{3, 4, 7, 8, 10, 12, 13\}$	$\{2, 5, 8, 9, 10, 12, 13\}$
$\{3, 5, 6, 8, 11, 12, 13\}$	$\{2, 4, 6, 9, 11, 12, 13\}$
$\{6, 7, 8, 9, 11, 12, 13\}$	$\{1, 6, 8, 10, 11, 12, 13\}$

The example is different from the separate 30-block cover in `known30_control.json`. Both were checked independently on all 715 four-subsets in the preparation packet.

B Computational protocol and frozen identifiers

The accompanying supplement preserves the exact lower-bound source tree under `computation/source/research/horizonmath/c13_classified_pencils/`. Its `computation/reproduce.py` runs the portable checks from an unpacked copy without access to the private repository. The source snapshot is research-lab commit `fabd230929bf17b95d6c74d738f78c553c2e0216`. The two verifier entry points are `verify_pencils.py` and `independent_raw_replay.py`; the independent join is in `peer_union_join.py`. The frozen inputs and file hashes are recorded in `minimum_link_catalog.json`, `domain_reconstruction_receipt.json`, and `verification_manifest.json`. The manifest identifies both program inputs and replay outputs.

From that directory, the documented portable commands are:

```
python3 verify_pencils.py > primary.json
python3 -O verify_pencils.py > primary-O.json
python3 independent_raw_replay.py > independent.json
python3 -O independent_raw_replay.py > independent-O.json
cmp primary.json primary-O.json
cmp independent.json independent-O.json
python3 -m unittest -v test_classified_pencils.py
```

The historical production release gate additionally records a native C++ cross-check. The published-dependency audit is in `classification_dependency_lock.py`. The independent joining routine must count only *actually compatible* tuples; its reported Cartesian potential is not the number of checked tuples.

Pinned identity data (original research snapshot):

Item	Frozen identifier or exact invariant
Research-lab C13 checkpoint	`eec0acba559a732c98babfab497e884fc2fe9e20`
All root/fourset positions	33,660
Unpruned compatible tuples	280,152; every union has size 15
Lean entry module	`C13Global/Final.lean`
Lean fixed main theorem	`exact30_after_published_results`

The primary reconstruction domain's complete SHA-256 is the concatenation of the following two 32-character hexadecimal strings, in order:

```
425ec3b48e8eecdcc3eb5ac1dbe2555b
28420967ea7db00cb690c1e44ddbcb5e
```

The Lean entry module is in package `c13-covering-lean/global-census-20261010`. The table records existing-source evidence; it is not a claim that the entire private lower-bound computation or Lean package was newly re-executed for this manuscript revision. The accompanying dated audit records the fresh executions performed for this revision, separately from historical Lean assurance. The Lean package is not required to reproduce the computational proof presented in this paper.

C Literal classification input

For transcription checking, the Figure 1 core is the following list on $\{0, 1, \ldots, 9, A, B\}$, with $A = 10$ and $B = 11$:

```
014589  0167AB  2345AB  236789
02468A  02579B  13469B  13578A
03478B  03569A  12479A  12568B.
```

To this core add one six-set extending each of 0123, 4567 and $89AB$, choosing the other two points from the complementary eight. The program enumerates all 28^3 choices, including choices equivalent under relabelling. The construction group has order 576 and gives 67 representative completions. For completeness of the search it is sufficient that every literal completion is in one of these orbits; no claim that arbitrary cover isomorphisms preserve the core is needed.

For Figure 6 the labels are $0123456789TE$, with $T = 10$ and $E = 11$:

```
012345  012678  0129TE  0378TE  0456TE
045789  034569  13689E  14579E  14568T
13457T  23679T  24589T  24567E  23458E.
```

These are transcriptions of the cited classification inputs, not a new classification proof. Source-figure comparison and the applicability of the published classification must be checked independently of the code.

D Finite-verifier logic in pseudocode

The following specifies the logical search performed by the exact finite verifier, independently of implementation-level optimizations. Full reproducibility requires the released source and input catalog.

```
for each root link L in the 68-link catalog:
  for each Q among the 495 ordinary foursets:
    build each complete domain D[L,q], q in Q
    if some pair of domains has no compatible pair:  continue
    repeat:  discard values lacking support in another domain
    until no change
    if some domain is empty:  continue
    enumerate all four-tuples matching all six pair relations
    for each compatible tuple, measure its literal block union
    assert union cardinality exceeds 14
```

Arc-consistency deletion is valid by the induction in the proof of Proposition 4.1. Domain construction is exhaustive by Lemma 3.1, *provided* the cited published classification is correct and fully transcribed. The final join does not assume a solver's answer is valid merely because it says "infeasible."

References

[1] D. M. Gordon, O. Patashnik, J. Petro and H. Taylor, *Minimum* $(12, 6, 3)$ *Covers*, Ars Combinatoria **40** (1995), 161–177. https://combinatorialpress.com/ars-articles/volume-040-ars-articles/minimum-12-6-3-covers/.

[2] D. M. Gordon, *La Jolla Covering Repository*, $(v, k, t) = (13, 7, 4)$, historic upper and lower bounds. `https://ljcr.dmgordon.org/cover/table.html` and `https://ljcr.dmgordon.org/cover/low_tab.html`.

[3] E. Dwares, *Formal elimination of link class 52 in the* $C(13, 7, 4)$ *search*, public certification project (2026), `https://github.com/edwares/class52-formal-certification`. The certification concerns one class, not the full optimum.

[4] C. Krug, *The covering number* $C(12, 6, 4)$ *is 41*, preprint (2026), arXiv:2607.23766, `https://arxiv.org/abs/2607.23766`.